AI SOFTWARE ENGINEERING ASSISTANT

Technical Project Documentation

Domain: Software Development

Team Members: _____

Institution: _____

Course / Department: _____

Academic Year: 2026–2027

Source of Truth: Attached Google Colab Jupyter Notebook

Documentation status: Current notebook implementation only. Features are explicitly classified as Implemented, Prototype / Demonstration, or Future / Planned.

Project Status. Implementation Status Snapshot

The attached notebook contains a working multi-agent software-engineering prototype built around Gemini 2.5 Flash through LiteLLM and the OpenAI Agents SDK. It demonstrates structured agent outputs, code generation/review/debugging, file analysis, actual pytest execution, GitHub issue retrieval, a handoff definition, and session initialization. It does not demonstrate a single production-ready autonomous lifecycle with every component connected end-to-end.

Capability	Status	Notebook evidence / interpretation
Requirements Analysis Agent	Implemented	Agent with RequirementAnalysis Pydantic output; executed successfully on a FastAPI login issue.
Coding Assistant	Implemented	Agent generates CodeSolution; used for initial code and Version 2 improvement.
Code Reviewer	Implemented	Agent returns approval, issues and suggestions; initial and second reviews executed.
AI-generated Testing Agent	Implemented	Generates test cases, potential failures and a testing summary.
Actual pytest execution	Implemented	run_tests tool invokes pytest; notebook reports 4/4 tests passed.
Bug Investigation Agent	Implemented	Analyzes source, review issues and testing failures; returns structured bug analysis.
Version 2 improvement loop	Implemented	Coding Agent is called with bug findings and produces improved code.
Documentation Agent	Implemented	Produces project summary, changes, setup and usage instructions.
Code File Analyst	Implemented	Uses read_file and search_code tools to inspect a sample Python file.
GitHub Requirements Analysis Agent	Implemented	Uses GitHub REST API tool and successfully analyzes Flask issue #1.
File write tool	Implemented	write_file function exists with exception handling; no agent is shown using it in the notebook.
Agent handoff definition	Prototype / Demonstration	Handoff Requirements Agent points to Handoff Coding Agent.
Handoff execution	Prototype / Demonstration	Attempted execution failed with Gemini 2.5 Flash 429 quota exhaustion.
Session / memory	Prototype / Demonstration	OpenAIConversationsSession is initialized; notebook does not demonstrate a completed multi-turn run using it.
Single unified autonomous pipeline	Future / Planned	The notebook demonstrates separate workflows and a handoff prototype, not a production-grade unified orchestrator.

1. Cover Page

Project Title: AI Software Engineering Assistant

Domain: Software Development

Team Members: Placeholder

Institution: Placeholder

Academic Year: 2026–2027

2. Abstract

The AI Software Engineering Assistant is a notebook-based multi-agent prototype designed to support common software-engineering activities using large language models. The implementation uses Gemini 2.5 Flash through LiteLLM and the OpenAI Agents SDK. Specialized agents analyze requirements, generate code, review code, generate test cases, investigate bugs, improve code, and generate project documentation.

The notebook also demonstrates practical tool use. A Code File Analyst can read and search source files, a testing tool executes pytest, and a GitHub tool retrieves public issue information through the GitHub REST API. Structured Pydantic schemas are used for several agent outputs. An agent-handoff architecture and OpenAI conversational session are also present, but these are best classified as prototype/demonstration elements because the handoff execution encountered a Gemini quota error and session usage is not demonstrated as a completed multi-turn workflow.

3. Introduction

Modern software development involves repeated cycles of requirement interpretation, implementation, code review, testing, debugging and documentation. The notebook explores how these responsibilities can be decomposed into specialized AI agents rather than asking a single model to perform every task.

The implementation is deliberately modular: each agent has role-specific instructions and, where appropriate, a structured Pydantic output schema or access to a function tool. This makes the outputs easier to consume by subsequent stages and gives the project a clear architecture for future orchestration.

4. Problem Statement

Developers frequently spend time translating issue descriptions into technical requirements, proposing implementation changes, reviewing code for defects and security problems, preparing test cases, diagnosing failures and maintaining documentation. The project investigates an AI-assisted workflow that can support these activities through specialized agents and software tools.

5. Motivation

The motivation is to demonstrate practical application of agentic AI to software engineering. Instead of treating an LLM only as a conversational coding assistant, the project assigns explicit responsibilities to multiple agents and connects them with structured outputs and tools.

A second motivation is traceability: the notebook can show what a requirements agent produced, what the coding agent generated, what the reviewer identified, what the testing agent considered, and how the bug investigation stage influenced Version 2 code.

6. Objectives

- Analyze natural-language software problems and extract structured requirements.
- Generate a proposed implementation from analyzed requirements.
- Review generated code for correctness, bugs, security, readability, maintainability and edge cases.
- Generate test cases and potential failure scenarios.
- Diagnose bugs using source code, review findings and testing information.
- Generate an improved Version 2 implementation.
- Perform a second code review after improvement.
- Inspect project files using function tools.
- Retrieve and analyze public GitHub issues.
- Demonstrate agent handoffs and conversational-session support as extensible architecture.

7. Proposed Solution

The proposed solution is a collection of role-specialized agents running on a common Gemini 2.5 Flash model through LiteLLM. The OpenAI Agents SDK provides the Agent and Runner abstractions, structured outputs, function tools and handoff capabilities.

The core demonstrated chain is: Requirements Analysis → Coding → Code Review → AI-generated Testing → Bug Investigation → Coding for Version 2 → Second Code Review → Documentation. Separate demonstrations cover file analysis, GitHub issue analysis, actual pytest execution and agent handoff.

8. System Overview

The notebook runs in Google Colab. A Gemini API key is loaded from Colab userdata, and tracing is disabled. A LitellmModel is configured for gemini/gemini-2.5-flash. Agents are then instantiated with instructions, models, Pydantic output types and/or function tools.

The system is therefore best understood as an experimental multi-agent software-engineering workbench rather than a deployed software product. The notebook provides reproducible demonstrations of individual capabilities and selected workflows.

9. Functional Requirements

- Accept a software problem as natural-language input.
- Convert a problem into structured requirements and acceptance criteria.
- Generate proposed implementation code and affected-file information.
- Review source code and return approval, issues and suggestions.
- Generate normal, edge and invalid-input test cases.
- Analyze review/testing findings to identify bugs, root causes, severity and fixes.
- Generate an improved implementation using bug findings.
- Perform a second code review.
- Generate project/setup/usage documentation.
- Read and search source files through tools.

- Execute pytest through a tool.
- Fetch public GitHub issue information and analyze it.

10. Non-Functional Requirements

- Structured outputs should be machine-readable through Pydantic models where schemas are defined.
- Agents should have focused role instructions.
- Tools should return useful error messages when operations fail.
- The assistant should avoid inventing file contents or GitHub issue contents.
- Testing claims should be based on actual pytest output when the execution tool is used.
- The architecture should be modular enough to add or replace agents and tools.
- Security-sensitive recommendations should distinguish simulation/demo behavior from production requirements.

11. System Architecture

At the infrastructure level, the notebook contains four major layers: (1) execution environment and dependencies, (2) LLM/model layer, (3) agent layer, and (4) tool/integration layer. Google Colab supplies the Python execution environment and secret storage. The OpenAI Agents SDK supplies Agent and Runner. LiteLLM connects the SDK model abstraction to Gemini 2.5 Flash. Function tools provide controlled access to files, pytest and GitHub.

Architecture status: Implemented at component level. The complete architecture is not demonstrated as one continuously running production pipeline.

```
flowchart TB
    U[Developer / User] --> A[OpenAI Agents SDK]
    A --> M[LiteLLM Model Adapter]
    M --> G[Gemini 2.5 Flash]
    A --> AG[Specialized Agents]
    AG --> T[Function Tools]
    T --> F[Project Files]
    T --> P[pytest]
    T --> GH[GitHub REST API]
    A --> H[Handoff Mechanism]
    A --> S[Session / Memory Support]
```

12. Agent Architecture

Agent	Purpose	Input	Output	Tools / Interaction	Status
Requirements Analysis Agent	Extract problem, expected behavior, component, technical requirements and acceptance criteria.	Natural-language software problem	RequirementAnalysis	No function tools; feeds Coding Assistant in demonstrated manual chain.	Implemented
Coding Assistant	Propose or improve implementation.	Structured requirements or bug-fix prompt	CodeSolution	No function tools; consumes requirements and bug analysis.	Implemented
Code Reviewer	Evaluate correctness, bugs, security, readability, maintainability and edge cases.	Source code	ReviewResult	No function tools; used twice in lifecycle.	Implemented
Testing Agent	Generate tests, edge cases and potential failures; estimate whether tests pass.	Source code + review	TestingResult	One version is reasoning-only; a separate version has run_tests tool.	Implemented
Bug Investigation Agent	Diagnose bugs/vulnerabilities and root causes; recommend fixes.	Source + review issues + testing failures	BugAnalysis	No function tools; feeds Version 2 Coding call.	Implemented
Documentation Agent	Generate summary, changes, setup and usage documentation suitable for README.	Final code	DocumentationResult	No file-writing use demonstrated.	Implemented
GitHub Requirements Analysis Agent	Fetch and analyze public GitHub issues.	Owner/repo/issue number	Natural-language requirements analysis	get_github_issue	Implemented
Code File Analyst	Inspect source files and explain problems.	File path + user task	Natural-language analysis	read_file, search_code	Implemented
Handoff Requirements Agent	Analyze requirements and hand task to Handoff Coding Agent.	Natural-language software problem	Handoff-driven final response	handoffs=[Handoff Coding Agent]	Prototype / Demonstration
Handoff Coding Agent	Receive requirements through handoff and propose implementation.	Handoff context	Implementation proposal	No tools	Prototype / Demonstration

Requirements Analysis Agent

- Purpose: translate a software problem into structured engineering requirements.
- Input: natural-language problem statement.
- Processing: follows explicit analyst instructions and returns a RequirementAnalysis Pydantic object.
- Output fields: problem, expected_behavior, affected_component, technical_requirements, acceptance_criteria.
- Interaction: the notebook passes the structured result to the Coding Assistant.

Coding Assistant

- Purpose: propose implementation code from requirements and later produce Version 2 from bug findings.
- Input: serialized requirements for Version 1; source code plus bug findings for Version 2.
- Output: CodeSolution containing explanation, code and files_affected.
- Interaction: receives information from Requirements Analysis and Bug Investigation stages.

Code Reviewer

- Purpose: assess correctness, bugs, security, readability, maintainability and edge cases.
- Output: ReviewResult with approved, issues and suggestions.
- The notebook executes it twice: once on Version 1 and once on Version 2.

Testing Agent

- Purpose: generate test cases and identify potential failures.
- Output: TestingResult with test_cases, potential_failures, all_tests_passed and testing_summary.
- A separate tool-enabled Testing Agent can execute actual pytest and is instructed never to claim a test passed without tool confirmation.

Bug Investigation Agent

- Purpose: combine source code, review issues and testing failures into a diagnosis.
- Output: BugAnalysis with bugs_found, root_causes, severity and recommended_fixes.
- Its findings are fed into the Coding Assistant to generate Version 2.

Documentation Agent

- Purpose: summarize the final software solution for README-style documentation.
- Output: DocumentationResult containing project_summary, changes_made, setup_instructions and usage_instructions.
- Important limitation: it does not automatically generate this full 36-section capstone report.

GitHub Requirements Analysis Agent

- Purpose: retrieve a public GitHub issue and extract the problem, expected behavior, affected components and technical requirements.
- Tool: get_github_issue uses requests.get against the public GitHub REST endpoint.
- Demonstration: Flask issue #1 was successfully retrieved and analyzed.

Code File Analyst

- Purpose: inspect a real project file rather than relying only on pasted code.
- Tools: read_file and search_code.
- Demonstration: sample_app.py was created and analyzed for hardcoded credentials, plaintext password handling, weak password and lack of brute-force protection.

Handoff Agents

- Handoff Requirements Agent is configured with handoffs=[Handoff Coding Agent].
- The intended behavior is requirements analysis followed by transfer to the coding specialist.
- The notebook attempts execution, but the call fails because Gemini 2.5 Flash returns a 429 free-tier quota error. Therefore this is classified as Prototype / Demonstration rather than a fully validated handoff workflow.

13. Detailed Workflow

Core demonstrated lifecycle. The notebook manually chains several Runner.run calls. It is not a single orchestrator that automatically invokes every stage.

```
flowchart LR
    I[User Problem] --> R[Requirements Analysis]
    R --> C[Coding Assistant]
    C --> V1[Version 1 Code]
    V1 --> CR1[Code Review]
    CR1 --> T[AI Test Analysis]
    T --> B[Bug Investigation]
    B --> C2[Coding Assistant - Version 2]
    C2 --> V2[Improved Code]
    V2 --> CR2[Second Code Review]
    CR2 --> D[Documentation Agent]
```

GitHub workflow. The GitHub agent receives owner, repository and issue number, calls get_github_issue, extracts issue number/title/state/description, and asks Gemini to analyze the retrieved issue.

```
flowchart LR
    U[Repo + Issue Number] --> GH[get_github_issue]
    GH --> API[GitHub REST API]
    API --> DATA[Issue JSON]
    DATA --> GHA[GitHub Requirements Analysis Agent]
    GHA --> REQ[Problem / Expected Behavior / Components / Requirements]
```

File-analysis workflow. The Code File Analyst can call read_file and search_code. The demonstration creates sample_app.py, asks the agent to inspect it, and the agent identifies security problems from the actual file contents.

```
flowchart LR
    U[File Analysis Request] --> FA[Code File Analyst]
    FA --> RF[read_file]
    FA --> SC[search_code]
    RF --> FILE[Project File]
    SC --> FILE
    FA --> O[Security / Code Findings]
```

Testing workflow. There are two distinct testing demonstrations: an LLM reasoning agent generates candidate test cases, while the tool-enabled Testing Agent invokes pytest through subprocess. The latter is the source of the actual '4 passed, 0 failed' result.

```
flowchart LR
    CODE[Source Code] --> TA[AI Testing Agent]
    REVIEW[Code Review] --> TA
    TA --> CASES[Generated Test Cases]

    TF[Test File] --> RT[run_tests]
    RT --> PY[python -m pytest]
    PY --> RESULT[Actual Test Output]
    RESULT --> TA2[Tool-enabled Testing Agent]
```

Handoff workflow. The handoff is configured in the Agent definition. The requirements agent can transfer control to the coding agent. The attempted runtime demonstration failed because the Gemini API quota was exhausted, so successful end-to-end handoff behavior is not claimed.

14. Technologies Used

Technology / Library	Actual role in notebook	Status
Python	Primary programming language and Colab runtime.	Implemented
Google Colab	Execution environment and secret storage through userdata.	Implemented
openai-agents[litellm]	Agents, Runner, function tools, model integration and handoffs.	Implemented
LiteLLM	Model adapter used through LitellmModel.	Implemented
Gemini 2.5 Flash	LLM used by the agents.	Implemented
Pydantic	Structured output schemas for agent results.	Implemented
requests	GitHub REST API call.	Implemented
pytest	Actual test execution tool.	Implemented
subprocess	Runs pytest from Python tool.	Implemented
bcrypt	Used in generated Version 2 FastAPI example code.	Demonstrated in generated code
OpenAI client / Conversations Session	Session/memory support initialization and client initialization.	Prototype / Demonstration

15. AI/LLM Architecture

Gemini. The notebook configures gemini/gemini-2.5-flash as the model for the agents. The Gemini API key is loaded from Google Colab userdata.

LiteLLM. LitellmModel provides the model adapter used by the OpenAI Agents SDK to call Gemini. This separates the agent framework from the underlying provider interface.

OpenAI Agents SDK. Agent defines role, instructions, model, output schema, tools and handoffs. Runner.run executes an agent with an input.

Structured outputs. Several agents specify output_type using Pydantic models. This constrains the expected shape of outputs and makes downstream processing predictable.

Tools. Function tools expose deterministic operations such as reading files, searching code, executing pytest and retrieving GitHub issues.

Handoffs. The notebook uses the handoffs parameter to connect the Requirements Handoff Agent to the Handoff Coding Agent.

Sessions / memory. OpenAIConversationsSession is instantiated, and an OpenAI client/API key is initialized. The notebook does not show a completed Runner.run call passing the session object through a multi-turn workflow, so persistent conversational memory should be described as a prototype/demonstration capability.

16. Tools and Integrations

Tool	Implementation	Use	Status
read_file	open(file_path, 'r', encoding='utf-8')	Read project file contents.	Implemented
write_file	open(file_path, 'w', encoding='utf-8')	Write content to a project file.	Implemented; not shown being invoked by an agent
search_code	Read lines and case-insensitive substring search.	Locate relevant code lines.	Implemented
run_tests	subprocess.run(['python','-m','pytest',test_file,'-q']) with 30-second timeout.	Execute actual tests.	Implemented and successfully demonstrated
get_github_issue	requests.get('https://api.github.com/repos/{owner}/{repo}/issues/{issue_number}')	Fetch public issue metadata/body.	Implemented and successfully demonstrated

17. Data Flow

The primary data flow is text and structured Python objects. The Requirements Agent returns a Pydantic object. The notebook then serializes it with model_dump() before passing it to the Coding Assistant. Review and testing objects are similarly converted into text when supplied to later agents. The Bug Investigation Agent consumes source code plus review issues and testing failures. Its structured output is inserted into the Version 2 coding prompt.

```
flowchart TD
    U[Problem / Issue] --> R[RequirementAnalysis]
    R -->|model_dump| C[Coding Assistant]
    C --> CS[CodeSolution]
    CS --> RV[ReviewResult]
    CS --> TS[TestingResult]
    RV --> B[BugAnalysis]
    TS --> B
    B --> C2[Version 2 CodeSolution]
    C2 --> RV2[Second ReviewResult]
    RV2 --> D[DocumentationResult]
```

18. Input and Output Formats

Inputs are primarily plain strings supplied to Runner.run. Tool inputs are typed function parameters such as file_path, search_term, owner, repo and issue_number. Structured agent outputs use Pydantic BaseModel classes.

Component	Input	Output
Requirements Agent	Problem text	RequirementAnalysis
Coding Assistant	RequirementAnalysis dump / bug-fix prompt	CodeSolution

Component	Input	Output
Code Reviewer	Source code	ReviewResult
Testing Agent	Source + review	TestingResult
Bug Agent	Source + review issues + testing failures	BugAnalysis
Documentation Agent	Final code	DocumentationResult
Code File Analyst	File task	Natural-language analysis using tools
GitHub Agent	Owner + repository + issue number	Issue analysis
run_tests	Test filename	Exit code + pytest output

19. Implementation Details

The notebook begins by installing openai-agents[litellm], loading GEMINI_API_KEY from Colab userdata, importing Agent/Runner/LitellmModel, disabling tracing, and configuring Gemini 2.5 Flash.

The implementation then develops the core agents sequentially. Each stage has a Pydantic schema where structured output is required. The notebook prints outputs to make each stage observable during a capstone demonstration.

Later cells add deterministic tools. These tools are decorated with function_tool so the Agents SDK can expose them to agents. The notebook then demonstrates file inspection, actual pytest execution and GitHub issue retrieval.

20. Code Explanation

Core model configuration

```
gemini_model = LitellmModel(
    model="gemini/gemini-2.5-flash",
    api_key=os.environ["GEMINI_API_KEY"]
)
```

Structured requirement schema

```
class RequirementAnalysis(BaseModel):
    problem: str
    expected_behavior: str
    affected_component: str
    technical_requirements: List[str]
    acceptance_criteria: List[str]
```

Tool example: pytest execution

```
@function_tool
def run_tests(test_file: str) -> str:
    result = subprocess.run(
        ["python", "-m", "pytest", test_file, "-q"],
        capture_output=True,
        text=True,
        timeout=30
    )
    return ...
```

Handoff definition

```
handoff_requirements_agent = Agent(
    name="Handoff Requirements Agent",
    ...
    handoffs=[handoff_coding_agent]
)
```

The code is intentionally not reproduced in full. The important architectural pattern is role-specific instructions + optional Pydantic output + optional function tools + optional handoffs.

21. Testing Strategy

The notebook demonstrates two levels of testing support. First, the AI Testing Agent reasons over source code and review results to generate test cases and potential failures. This is an analytical output and is not equivalent to executing the tests.

Second, the notebook creates sample_app.py and test_sample_app.py, defines run_tests using subprocess, and runs pytest. The tool-enabled Testing Agent reports that all 4 tests passed and 0 failed. This is the strongest evidence of actual test execution in the notebook.

The four executed tests cover valid login, wrong password, wrong username and empty password for the simple sample_app.py function.

22. Bug Detection and Debugging Strategy

The Bug Investigation Agent combines three information sources: source code, code-review issues and testing failures. It returns a structured list of bugs, root causes, overall severity and recommended fixes.

In the demonstrated example, the agent identified hardcoded credentials, insufficient whitespace handling, brittle Pydantic error-type inspection, hardcoded error messages and an inconsistent definition of an empty password. The overall severity was reported as High.

The resulting bug analysis is then included in a new coding prompt to create Version 2. This is an example of iterative AI-assisted debugging rather than deterministic static analysis.

23. Code Review Strategy

The Code Reviewer uses explicit criteria: correctness, bugs, security issues, readability, maintainability and edge cases. It returns an approval boolean plus issue and suggestion lists.

The first review approved Version 1 but still identified security and maintainability issues. The second review approved Version 2 and provided production-oriented suggestions including username validation, rate limiting, a real JWT instead of a fake token and cleaner exception handling.

24. GitHub Issue Analysis

The GitHub integration is implemented through requests. The get_github_issue tool constructs a public API endpoint from owner, repo and issue_number, performs a GET request with a 15-second timeout, checks for status 200, parses JSON and returns issue number, title, state and description.

The GitHub Requirements Analysis Agent is instructed to always use this tool when repository issue information is supplied and not to invent issue contents. The notebook demonstrates the workflow using pallets/flask issue #1 and produces a requirements-oriented explanation about dotted endpoint names.

Limitation: the demonstrated tool fetches a public issue only; there is no implementation for creating, updating, commenting on or closing issues, and no authenticated GitHub workflow is shown.

25. Agent Handoff Mechanism

The handoff prototype defines Handoff Coding Agent and Handoff Requirements Agent. The latter has handoffs=[handoff_coding_agent], allowing control to be transferred after requirements analysis.

The notebook attempts a handoff run for the FastAPI password problem. That execution fails with a LiteLLM/Gemini RateLimitError: HTTP 429, RESOURCE_EXHAUSTED, because the free-tier generate_content request quota for gemini-2.5-flash was exceeded. The notebook therefore proves the handoff configuration exists, but does not prove a successful end-to-end runtime handoff under that execution.

The notebook subsequently prints handoff_result.last_agent.name, which is shown as Handoff Coding Agent in the saved output context, but the actual run cell itself contains the quota error. For a capstone report, the safest

classification is Prototype / Demonstration and the quota failure should be explicitly disclosed.

26. Memory / Session Management

The notebook imports OpenAIConversationsSession and initializes chat_history = OpenAIConversationsSession(). It also attempts to initialize an AsyncOpenAI client and later loads OPENAI_API_KEY from Colab userdata if necessary.

This demonstrates awareness of conversational session infrastructure. However, the notebook does not show the chat_history object being supplied to Runner.run for a complete multi-turn conversation. Therefore, persistent memory should be described as a prototype/demonstration capability, not as a fully validated memory subsystem.

27. Security Considerations

- API keys are loaded from Colab userdata rather than being written directly into source code.
- The initial generated FastAPI example contains hardcoded demonstration credentials; the Code Reviewer correctly identifies this as a security vulnerability.
- The Bug Investigation Agent recommends hashed password storage and the Version 2 generated example uses bcrypt in a simulated user database.
- The generated Version 2 still returns a fake JWT token; the second reviewer identifies this as a production limitation.
- Rate limiting is recommended by the second reviewer but is not implemented.
- The file tools accept arbitrary file paths; a production implementation would need path restrictions, sandboxing and authorization.
- The GitHub tool uses public API access and does not implement authenticated write operations.
- LLM outputs should not be treated as automatically trustworthy code; review and execution boundaries remain necessary.

28. Error Handling

The file tools use try/except and return readable error strings when file operations fail. The GitHub tool catches exceptions and separately reports non-200 HTTP responses. The pytest tool catches exceptions and has a 30-second subprocess timeout.

The notebook itself also demonstrates handling of OpenAI client initialization errors and missing OpenAI API keys through try/except logic.

The most important observed external-service failure is the Gemini 429 quota error during the handoff demonstration. This is not silently hidden; it appears as a RateLimitError propagated through LiteLLM. This should be discussed during viva as an example of operational dependency on provider quotas.

29. Sample Use Cases

- Analyze a developer-reported API bug and turn it into acceptance criteria.
- Generate a proposed code fix from structured requirements.
- Review generated code for security and maintainability issues.
- Generate edge-case test scenarios.
- Diagnose bugs using review and test information.
- Generate a second implementation after diagnosis.
- Inspect an existing source file for security issues.
- Retrieve a public GitHub issue and convert it into software requirements.

- Execute a Python pytest file and report actual pass/fail results.
- Generate README-style documentation from the final code.

30. Sample Input / Output

Example 1 — Requirements Analysis

Input: “My FastAPI login endpoint crashes when the password field is empty.”

Output summary: the agent identified the FastAPI login endpoint and validation logic as affected, recommended password validation and HTTP 400 handling, and generated acceptance criteria for missing/empty password behavior.

Example 2 — Code Review

Version 1 review: Approved = True, but issues included hardcoded credentials and maintainability concerns around custom validation handling.

Example 3 — Actual Testing

pytest result: 4 tests passed, 0 failed for test_sample_app.py.

Example 4 — GitHub Issue

Input: owner=pallets, repo=flask, issue_number=1. The tool retrieves the public issue and the agent explains the requirement around dotted endpoint names.

31. Advantages

- Clear separation of responsibilities across specialized agents.
- Structured Pydantic outputs make downstream processing easier.
- Function tools connect LLM reasoning to deterministic software operations.
- Actual pytest execution prevents purely speculative claims about test results.
- GitHub integration makes issue analysis practical.
- Iterative review → diagnosis → Version 2 demonstrates an engineering feedback loop.
- The architecture is extensible through additional agents, tools and handoffs.
- The notebook is suitable for demonstrating agentic AI concepts during a capstone viva.

32. Limitations

- The project is a Google Colab prototype rather than a deployed production service.
- The agents primarily communicate through manually constructed prompt strings in notebook cells rather than a centralized workflow engine.
- The write_file tool is implemented but not demonstrated as an autonomous documentation/code modification action.
- The GitHub integration is read-only in the demonstrated implementation.
- The handoff execution encountered Gemini quota exhaustion.
- Session/memory infrastructure is initialized but not demonstrated through a completed multi-turn session workflow.
- The actual pytest demonstration targets a small sample_app.py, not the generated FastAPI Version 2 application.
- The LLM can propose insecure or incorrect code, so deterministic validation and human review remain important.
- No deployment, authentication layer for the assistant itself, observability dashboard, persistent database, CI/CD integration or repository-wide static analysis is demonstrated.

33. Future Enhancements

- Build a single orchestrator that automatically executes the complete lifecycle.
- Use successful handoffs between all relevant specialist agents.
- Connect session objects to actual multi-turn runs and persistent project context.
- Allow agents to safely write changes through sandboxed file tools.
- Add repository-wide code search and directory discovery.
- Integrate authenticated GitHub operations such as comments, labels and pull requests.
- Generate and execute tests against the generated implementation, not only a separate sample application.
- Add static analysis and linting tools alongside LLM review.
- Add CI/CD integration and automated pull-request review.
- Add rate-limit handling, retries, model fallback and provider failover.
- Replace fake authentication tokens with real JWT/OAuth implementations in example applications.
- Add a user-facing web interface and project dashboard.

34. Results / Expected Results

Observed results in the notebook: Requirements analysis, coding, initial review, AI test analysis, bug investigation, Version 2 generation, second review, documentation generation, file analysis and GitHub issue analysis all have successful saved executions. Actual pytest execution reports four passed tests.

Prototype results: Handoff configuration and session infrastructure are present. The attempted handoff execution hit the Gemini free-tier quota limit, so it should not be presented as a fully validated runtime workflow.

Expected future result: With orchestration, persistent session context, safer file modification and reliable model/provider management, the architecture could evolve into a repository-aware software engineering assistant.

35. Conclusion

The notebook successfully demonstrates the core idea of an AI Software Engineering Assistant: software-engineering responsibilities can be decomposed into specialized agents with structured outputs and deterministic tools. The most convincing demonstrations are the requirements-to-code-to-review-to-debugging loop, real file analysis, actual pytest execution and GitHub issue retrieval.

The project should be presented honestly as a capable prototype. Its strongest capstone value lies in the architecture and demonstrated agentic patterns, while production orchestration, persistent memory, authenticated GitHub operations, safe repository modification and robust provider handling remain future work.

36. References

Primary source: Attached Google Colab notebook, “sdk final capstone project (2).ipynb”. All implementation-specific statements in this report are derived from that notebook.

OpenAI Agents SDK documentation — used conceptually for Agent, Runner, tools, handoffs and sessions.

LiteLLM documentation — used for the model adapter demonstrated in the notebook.

Google Gemini API documentation — relevant to Gemini 2.5 Flash and quota behavior observed during execution.

Pydantic documentation — relevant to BaseModel and structured validation.

pytest documentation — relevant to actual test execution.

GitHub REST API documentation — relevant to public issue retrieval.

These external references are listed for technical context; the attached notebook remains the primary source of truth for what this project actually implements.

Mermaid Diagrams

The following Mermaid source represents the current notebook architecture. The diagrams deliberately avoid implying that every component is one fully automated production pipeline.

Overall System Architecture

```
flowchart TB
    U[Developer / User]
    U --> SDK[OpenAI Agents SDK]
    SDK --> L[LiteLLM]
    L --> G[Gemini 2.5 Flash]
    SDK --> A[Specialized Agents]
    A --> T[Function Tools]
    T --> FILES[Project Files]
    T --> PY[pytest]
    T --> GH[GitHub REST API]
    SDK --> H[Handoff Prototype]
    SDK --> S[Session / Memory Prototype]
```

Multi-Agent Architecture

```
flowchart TD
    R[Requirements Analysis Agent]
    C[Coding Assistant]
    CR[Code Reviewer]
    T[Testing Agent]
    B[Bug Investigation Agent]
    D[Documentation Agent]
    GH[GitHub Requirements Analysis Agent]
    F[Code File Analyst]
    HR[Handoff Requirements Agent]
    HC[Handoff Coding Agent]

    R --> C
    C --> CR
    CR --> T
    T --> B
    B --> C
    C --> CR
    CR --> D
    GH --> R
    F --> CR
    HR -. handoff .-> HC
```

Agent Workflow

```
flowchart LR
    I[Issue / Problem]
    I --> R[Requirements]
    R --> C[Coding]
    C --> CR1[Review 1]
    CR1 --> T[Testing Analysis]
    T --> B[Bug Investigation]
    B --> V2[Version 2]
    V2 --> CR2[Review 2]
    CR2 --> D[Documentation]
```

GitHub Issue Workflow

```
flowchart LR
    INPUT[Owner + Repo + Issue Number]
    INPUT --> TOOL[get_github_issue]
    TOOL --> API[GitHub REST API]
    API --> JSON[Issue JSON]
    JSON --> AGENT[GitHub Requirements Agent]
    AGENT --> REQ[Requirements Analysis]
```

Code Review and Testing Workflow

```
flowchart TD
    CODE[Generated Source Code] --> REVIEW[Code Reviewer]
    REVIEW --> ISSUES[Review Issues]
    CODE --> TESTAI[AI Testing Agent]
    REVIEW --> TESTAI
    TESTAI --> CASES[Generated Test Cases]

    FILE[Test File] --> EXEC[run_tests]
    EXEC --> PY[pytest]
    PY --> RESULT[Actual Test Result]
```

Tool Integration Architecture

```
flowchart TB
    AG[Agents]
    AG --> RF[read_file]
    AG --> WF[write_file]
    AG --> SC[search_code]
    AG --> RT[run_tests]
    AG --> GI[get_github_issue]

    RF --> FS[Local Files]
    WF --> FS
    SC --> FS
    RT --> PT[pytest]
    GI --> GH[GitHub API]
```

Data Flow Diagram

```
flowchart TD
    INPUT[Problem / Issue] --> REQ[RequirementAnalysis]
    REQ --> CODE[CodeSolution]
    CODE --> REVIEW[ReviewResult]
    CODE --> TEST[TestingResult]
    REVIEW --> BUG[BugAnalysis]
    TEST --> BUG
    BUG --> CODE2[Improved CodeSolution]
    CODE2 --> REVIEW2[Second ReviewResult]
    REVIEW2 --> DOC[DocumentationResult]
```

A. Executive Summary

The AI Software Engineering Assistant is a multi-agent software-development prototype implemented in Google Colab. Gemini 2.5 Flash is accessed through LiteLLM and the OpenAI Agents SDK. The project demonstrates specialized agents for requirements analysis, coding, code review, testing analysis, bug investigation and documentation, plus file analysis and GitHub issue analysis tools.

The key technical contribution is the combination of LLM agents with structured Pydantic outputs and deterministic tools. The project also demonstrates an iterative feedback loop in which review and bug findings are passed back to the coding agent to produce Version 2, followed by a second review.

For evaluation, describe the project as a prototype with several working demonstrations. Do not claim that handoffs, persistent memory or a complete autonomous lifecycle are production-ready.

B. 2-Minute Viva Explanation

My project is an AI Software Engineering Assistant built as a multi-agent prototype in Google Colab. The goal is to help developers across the software lifecycle, from understanding an issue to generating code, reviewing it, testing it, debugging it and documenting it.

I use Gemini 2.5 Flash through LiteLLM with the OpenAI Agents SDK. Instead of one agent doing everything, I created specialized agents. The Requirements Agent converts a problem into structured requirements using Pydantic. The Coding Assistant generates code. The Code Reviewer checks correctness, security and maintainability. The Testing Agent generates test cases, and I also created a real pytest execution tool. The Bug Investigation Agent combines review and testing findings and recommends fixes. Then the Coding Assistant creates Version 2 and the reviewer checks it again. Finally, the Documentation Agent generates README-style documentation.

I also implemented a Code File Analyst with read and search tools and a GitHub Requirements Agent that retrieves public issues through the GitHub REST API. I have a handoff prototype and session support, but I present those honestly as prototype features because the handoff execution encountered a Gemini quota error and persistent session use was not fully demonstrated.

C. 5-Minute Viva Explanation

Start with the problem: software engineers repeatedly perform requirement interpretation, coding, review, testing, debugging and documentation. A single LLM can do these tasks, but specialized agents provide clearer responsibilities and structured interfaces.

The model layer is Gemini 2.5 Flash, accessed through LiteLLM. The orchestration abstraction comes from the OpenAI Agents SDK. Agent instructions define the role, while Pydantic output schemas define structured results.

The main workflow starts with a software problem. Requirements Analysis produces problem, expected behavior, affected component, technical requirements and acceptance criteria. The Coding Assistant consumes those requirements and returns explanation, code and affected files. The Code Reviewer checks the generated code. The Testing Agent produces test cases and possible failures. The Bug Investigation Agent combines code, review findings and testing information. Those findings are passed to the Coding Assistant to generate Version 2. Version 2 is reviewed again and then sent to the Documentation Agent.

The project also demonstrates tool use. read_file reads source files, search_code searches source lines, write_file can write content, run_tests executes pytest through subprocess, and get_github_issue calls the GitHub REST API. The GitHub agent successfully analyzes a Flask issue.

The important distinction in my project is between AI reasoning and deterministic execution. A generated test case is only a proposal; the run_tests tool actually executes pytest. The notebook reports four tests passed and zero failed for the sample application.

Finally, I have explored agent handoffs and conversational sessions. These are architecture extensions rather than fully validated production features. The handoff execution hit a Gemini 429 quota error, which is itself an important

operational limitation. Future work would add an orchestrator, persistent memory, safe repository editing, CI/CD and authenticated GitHub workflows.

D. 15 Likely Viva Questions with Answers

Question	Answer
1. Why use multiple agents instead of one agent?	Each agent has a focused responsibility and output contract. This improves modularity, makes prompts easier to reason about, and allows different tools or schemas per role.
2. What is LiteLLM doing?	LiteLLM provides the model adapter used by LitellmModel so the Agents SDK can invoke Gemini through a common interface.
3. Why Pydantic?	Pydantic defines structured output schemas. For example, RequirementAnalysis guarantees fields such as problem, expected_behavior and acceptance_criteria.
4. What is Runner.run?	It executes an agent with a supplied input and returns the run result, including the final output and agent information.
5. What is a function tool?	It exposes deterministic Python functionality to an agent. In this project the tools include file reading, code searching, pytest execution and GitHub issue retrieval.
6. What is the difference between AI testing and pytest?	AI testing generates or reasons about test cases. pytest actually executes a test file. Only the pytest result is evidence that those tests were executed.
7. What did the actual pytest demonstration show?	The notebook reports four tests passed and zero failed for test_sample_app.py.
8. How does GitHub integration work?	get_github_issue constructs the public GitHub REST endpoint, performs a requests GET, parses the JSON and returns issue number, title, state and description to the GitHub Requirements Agent.
9. What does the Requirements Agent output?	A Pydantic RequirementAnalysis object containing the problem, expected behavior, affected component, technical requirements and acceptance criteria.
10. How does bug investigation work?	The Bug Investigation Agent receives source code, code-review issues and testing failures, then returns bugs, root causes, severity and recommended fixes.
11. How is Version 2 generated?	The original code plus bug findings, root causes and recommended fixes are inserted into a new prompt to the Coding Assistant.
12. What is a handoff?	A handoff transfers control from one agent to another. The notebook configures Handoff Requirements Agent to hand off to Handoff Coding Agent.
13. Was the handoff fully successful?	No. The handoff is implemented as a prototype, but the demonstrated execution encountered a Gemini 2.5 Flash 429 quota exhaustion error.
14. Is memory fully implemented?	Session infrastructure is initialized using OpenAIConversationsSession, but a complete multi-turn Runner.run using that session is not demonstrated. Therefore it is prototype-level.
15. What is the biggest future improvement?	Build a single robust orchestrator that connects issue intake, repository context, coding, review, executable tests, debugging, safe file changes, GitHub operations and persistent session context into one controlled lifecycle.

E. Future Scope

- Repository-aware autonomous planning.
- Automatic creation of branches and pull requests.
- Authenticated GitHub comments and issue updates.
- Safe sandboxed code modifications through write tools.
- Actual execution of generated tests against generated code.
- Static analysis, linting and security scanners.
- CI/CD integration.
- Persistent project memory.
- Model fallback and quota-aware retries.
- Web-based developer dashboard.

- Approval gates before modifying or merging code.
- Traceable agent execution logs and evaluation metrics.

F. Key Technical Points to Remember for the Presentation

- Gemini 2.5 Flash is the LLM; LiteLLM is the adapter; OpenAI Agents SDK provides the agent framework.
- Agent = role + instructions + model + optional output schema + optional tools + optional handoffs.
- Pydantic gives structured, typed outputs instead of relying only on free-form text.
- Function tools connect the LLM to deterministic operations.
- The Requirements → Coding → Review → Testing → Bug → Version 2 → Review loop is the core engineering demonstration.
- AI-generated test cases are not the same as executed tests.
- run_tests invokes python -m pytest and returns exit code plus stdout/stderr.
- GitHub integration is read-only in the notebook demonstration.
- File analysis uses read_file and search_code; write_file exists but is not demonstrated as an agent action.
- The Documentation Agent generates four categories: project summary, changes made, setup instructions and usage instructions.
- Handoffs and session/memory are prototypes in the current notebook.
- The Gemini 429 quota error is an operational limitation, not evidence that the architecture itself is invalid.
- Do not call the notebook a production-ready autonomous software engineer; call it a multi-agent prototype with working demonstrations.
- Always distinguish implemented, prototype/demonstration and future functionality during viva.